Encontro 4: Diretivas, Binding e Eventos

Sumário

Encontro 4: Diretivas, Binding e Eventos....1
1. Introdução....2
2. Diretivas: Dando Superpoderes ao HTML....2
   2.1. Renderização Condicional: `v-if` e `v-show`....2
   2.2. Renderização de Listas: `v-for`....4
3. Binding de Atributos: `v-bind`....5
4. Manipulação de Eventos: `v-on`....6
5. Comunicação Filho para Pai com $emit....7
   5.1. Exemplos....7

1. Introdução

Nos encontros anteriores, aprendemos sobre a estrutura fundamental dos componentes Vue. Agora, vamos explorar as ferramentas que tornam esses componentes verdadeiramente dinâmicos e interativos: as diretivas, o binding de dados e a manipulação de eventos.

2. Diretivas: Dando Superpoderes ao HTML

Diretivas são atributos especiais com o prefixo `v-` que aplicamos aos nossos elementos HTML para instruir o Vue a manipular o DOM de certas maneiras. Elas são a principal forma de aplicarmos lógica e reatividade ao nosso template.

2.1. Renderização Condicional: `v-if` e `v-show`

Muitas vezes, queremos mostrar ou esconder um elemento com base em alguma condição. O Vue nos oferece duas diretivas para isso:

- `v-if`: Remove ou insere o elemento e seus componentes filhos do DOM. Se a condição for falsa, o elemento literalmente não existirá na página. Ele também pode ser usado com `v-else` e `v-else-if` para criar blocos condicionais.
- `v-show`: Apenas alterna a propriedade `display` do CSS do elemento. O elemento sempre estará no DOM, mas pode estar invisível (`display: none`).

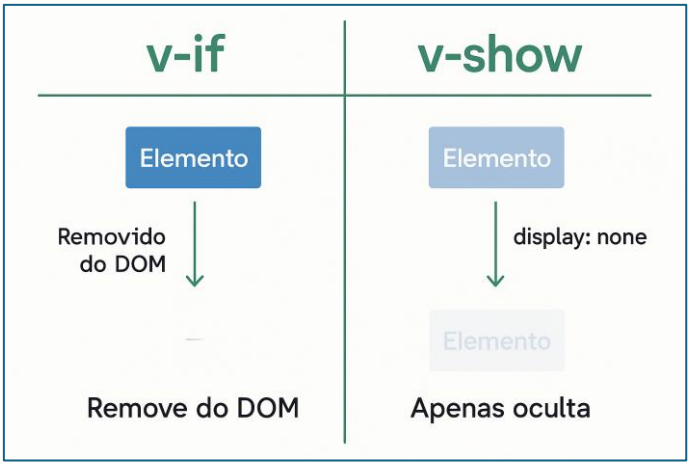


Figura 1: uso de `v-if` e `v-show`

A figura 1 ilustra o uso de `v-if` (removendo um elemento do DOM) e `v-show` (alterando a propriedade *display* para *none*).

Quando usar qual?

Diretiva	Custo de Alternância	Custo de Renderização Inicial	Caso de Uso Recomendado
`v-if`	Alto (recria o elemento)	Baixo (só renderiza se for true)	Quando a condição raramente muda. Ex: exibir um bloco para administradores.
`v-show`	Baixo (apenas CSS)	Alto (sempre renderiza o elemento)	Quando a condição muda frequentemente. Ex: abas de um menu.

Exemplo:

```
<template>
  <button @click="logado = !logado">Alternar Login</button>

  <!-- v-if: O h1 só será criado se 'logado' for true -->
  <h1 v-if="logado">Bem-vindo, Usuário!</h1>
  <p v-else>Você precisa fazer login para continuar.</p>

  <!-- v-show: O h2 sempre existe, mas pode estar escondido -->
  <h2 v-show="logado">Seu painel de controle</h2>
</template>

<script>
export default {
  data() {
    return {
      logado: false
    }
  }
}
</script>
```

2.2. Renderização de Listas: `v-for`

A diretiva `v-for` é usada para renderizar uma lista de itens com base em um array ou objeto. Ela itera sobre os dados e cria um elemento para cada item.

É obrigatório fornecer um atributo `:key` único para cada item da lista. A key ajuda o Vue a identificar cada nó, otimizando a performance ao reordenar, adicionar ou remover itens, evitando que o Vue tenha que redesenhar a lista inteira.



Figura 2: diretiva `v-for` itera sobre array e gera `<li>`

A figura 2 exibe a diretiva `v-for` iterando sobre um array de tarefas e gerando um elemento `<li>` para cada uma.

Exemplo:

```
<template>
  <div>
    <h2>Minhas Tarefas</h2>
    <ul>
      <!-- Itera sobre o array 'tarefas' -->
      <li v-for="tarefa in tarefas" :key="tarefa.id">
        {{ tarefa.texto }}
      </li>
    </ul>
  </div>
</template>

<script>
export default {
  data() {
    return {
      tarefas: [
        { id: 1, texto: 'Estudar Vue.js' },
        { id: 2, texto: 'Fazer o exercício prático' },
        { id: 3, texto: 'Tomar um café' }
      ]
    }
  }
}
</script>
```

3. Binding de Atributos: `v-bind`

Não podemos usar a interpolação (`{{ }}`) dentro de atributos HTML. Para vincular dinamicamente o valor de um atributo a uma propriedade do nosso componente, usamos a diretiva `v-bind`.

O `v-bind` é tão comum que possui um atalho: o caractere de dois-pontos (`:`).

- `v-bind:href="meuLink"` é o mesmo que `:href="meuLink"`
- `v-bind:class="minhaClasse"` é o mesmo que `:class="minhaClasse"`

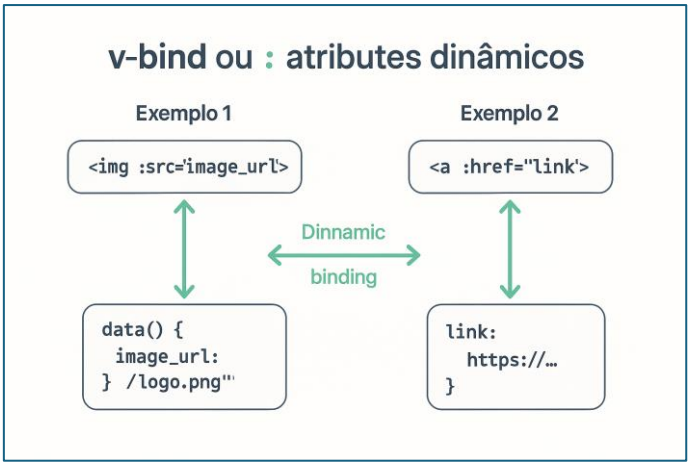


Figura 3: Exemplo da diretiva `v-bind`.

Na figura 3, a diretiva `v-bind` está sendo usada para popular dinamicamente o atributo `src` de uma imagem e o `href` de um link.

Exemplo:

```
<template>
  <div>
    <!-- Vincula o atributo 'src' à propriedade 'urlImagem' -->
    <img :src="urlImagem" alt="Logo do Vue">

    <!-- Vincula a classe CSS dinamicamente -->
    <p :class="{ 'texto-ativo': ativo, 'texto-grande': grande }">
      Este texto muda de classe.
    </p>
    <button @click="ativo = !ativo">Alternar Classe</button>
  </div>
</template>

<script>
export default {
  data() {
    return {
      urlImagem: '/logo.png',
      ativo: true,
      grande: true
    }
  }
}
</script>
```

4. Manipulação de Eventos: `v-on`

Para ouvir eventos do DOM (como cliques, envios de formulário, etc.) e executar um código quando eles ocorrem, usamos a diretiva `v-on`.

Assim como o `v-bind`, o `v-on` também tem um atalho: o caractere arroba (`@`).

- `v-on:click="meuMetodo"` é o mesmo que `@click="meuMetodo"`

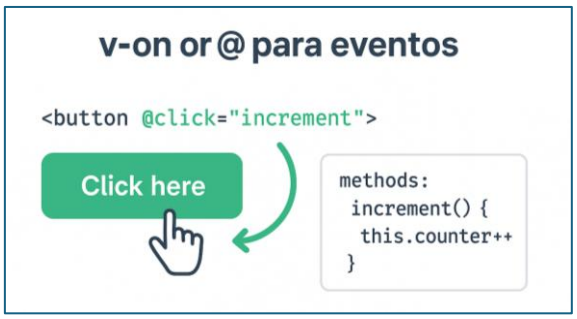


Figura 4: Uso da diretiva `v-on` ou `@click`

A figura 4 ilustra o uso da diretiva `v-on` ou `@click` em um botão, que chama um método no componente quando clicado.

Exemplo:

```
<template>
  <div>
    <p>Contador: {{ contador }}</p>

    <!-- Chama o método 'incrementar' ao clicar no botão -->
    <button @click="incrementar">Incrementar</button>

    <!-- Também podemos passar argumentos -->
    <button @click="somar(5)">Somar 5</button>
  </div>
</template>

<script>
export default {
  data() {
    return {
      contador: 0
    }
  },
  methods: {
    incrementar() {
      this.contador++;
    },
    somar(valor) {
      this.contador += valor;
    }
  }
}
</script>
```

5. Comunicação Filho para Pai com $emit

Já sabemos que os pais passam dados para os filhos através de `props`. Mas como um componente filho pode comunicar uma mudança ou um evento de volta para o pai? A resposta é com eventos customizados, usando o método `$emit`.

O fluxo é:

a) O componente filho emite um evento com um nome específico (e opcionalmente, um valor).
b) O componente pai ouve esse evento (usando `v-on` ou `@`) e executa um método em resposta.

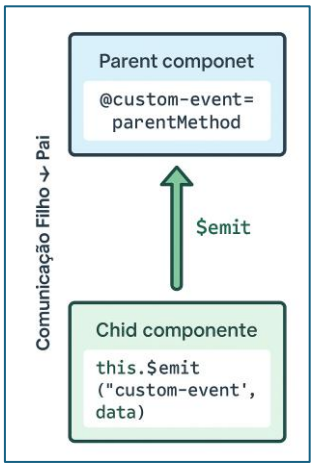


Figura 5: Fluxo de comunicação.

Na figura 5, o componente filho dispara um evento `$emit('evento-customizado')` para cima, e o componente pai o captura com `@evento-customizado="metodoDoPai"`.

5.1. Exemplos

a) Componente **BotaoAlerta.vue** (Filho)

```
<template>
  <button @click="dispararAlerta">Me clique!</button>
</template>

<script>
export default {
  name: 'BotaoAlerta',
  methods: {
    dispararAlerta() {
      // Emite um evento chamado 'alerta' com uma mensagem
      this.$emit('alerta', 'Esta mensagem veio do componente filho!');
    }
  }
}
</script>
```

b) Componente `App.vue` (Pai)

```vue
<template>
  <div id="app">
    <!-- Ouve o evento 'alerta' e chama o método 'mostrarMensagem' -->
    <BotaoAlerta @alerta="mostrarMensagem" />
    <p v-if="mensagem">Mensagem recebida: {{ mensagem }}</p>
  </div>
</template>

<script>
import BotaoAlerta from './components/BotaoAlerta.vue';

export default {
  name: 'App',
  components: { BotaoAlerta },
  data() {
    return {
      mensagem: ''
    }
  },
  methods: {
    // O payload do evento é recebido como argumento do método
    mostrarMensagem(payload) {
      this.mensagem = payload;
    }
  }
}
</script>
```

Com essas diretivas e padrões, você agora tem as ferramentas essenciais para construir aplicações Vue.js ricas e interativas.